

CC4302 Sistemas Operativos – Tarea 1 – Semestre Otoño 2025 – Profs.: Mateu, Torrealba, Arenas

Una función booleana (tipo *BoolFun*) recibe como parámetro un arreglo x de n variables booleanas (es decir cada variable puede ser solo verdadero o falso) y entrega un resultado booleano calculado a partir de las variables en x operadas con $\&\&$, $||$ y $!$. Por ejemplo f_4 es una función booleana que recibe un arreglo de 4 variables:

```
int f4(int x[]) {
    return (x[0]||!x[1]) && (!x[1]||x[2]||!x[3]);
}
```

La función *recuento* de más abajo evalúa una función booleana $f(x)$ para cada una de las 2^n combinaciones posibles de valores que pueden tomar las n variables del arreglo x , retornando el número de veces en que la función es verdadera. Esta función es lenta de calcular ya que toma tiempo $O(2^n)$.

```
typedef int (*BoolFun)(int x[]);
int cnt= 0;
void gen(int x[], int i, int n,
         BoolFun f) {
    if (i==n) {
        if ((*f)(x))
            cnt++;
    }
    else {
        x[i]= 0; gen(x, i+1, n, f);
        x[i]= 1; gen(x, i+1, n, f);
    }
}
int recuento(int n, BoolFun f, int p) {
    int x[n];
    gen(x, 0, n, f);
    return cnt;
}
```

Paralelice la función *recuento* de manera que se contabilice el número de veces en que f se hace verdadera usando 2^p threads, en donde p es el parámetro de la función *recuento* sin utilizar.

Restricción: Se requiere que el incremento de velocidad (*speed up*) sea al menos un factor 1.5x. Cuando pruebe su tarea en su computador asegúrese que posea al menos 2 cores, que esté configurado en modo alto rendimiento y que no estén corriendo otros procesos intensivos en uso de CPU al mismo tiempo. De otro modo podría no lograr el *speed up* solicitado.

Ayuda: La forma de crear los threads es muy similar a la manera en que se crearon los threads para resolver *quicksort* en paralelo en la clase auxiliar del miércoles 19 de marzo. En esta tarea no necesita y no debe usar ningún mutex y tampoco condiciones. Programe una función recursiva similar a la función *gen* del cuadro de más arriba, pero que agregue el parámetro p . Para llamadas en que $i \geq p$, el problema se debe resolver secuencialmente tal como lo hace

gen. Para llamadas en que $i < p$, Ud. debe generar un thread que ejecute en paralelo una de las llamadas recursivas. La complejidad de este problema es que los threads comparten la memoria y por lo tanto habrán dataraces con los múltiples threads modificando (i) el contador *cnt* y (ii) el arreglo x . Si intenta utilizar un mutex para evitar los dataraces, nunca va a obtener el *speed up* requerido. Para resolver (i) elimine la variable global *cnt*. Haga que la función *gen* retorne el contador, en vez de contabilizar en una variable global. Para solucionar (ii) haga que el thread reciba una copia del arreglo x . Así los threads no compartirán ninguna variable.

Instrucciones

Baje *tl.zip* de U-cursos y descomprímalo. El directorio *Tl* contiene los archivos *test-sat.c*, *Makefile*, *sat.h* (con los encabezados requeridos) y otros archivos. Ud. debe programar en el archivo *sat.c* la función *recuento*. Defina otras funciones si las necesita. **Se descontarán 5 décimas si su solución no usa la indentación de Kernighan.**

Pruebe su tarea bajo Debian 12. Ejecute el comando *make* sin parámetros. Le mostrará las opciones que tiene para compilar su tarea. Estos son los requerimientos para aprobar su tarea:

- *make run-san* debe felicitarlo y no reportar ningún incidente en el manejo de memoria.
- *make run-thr* debe felicitarlo y no reportar ningún datarace.
- *make run* debe felicitarlo por aprobar este modo de ejecución. El *speed up* reportado debe ser de al menos 1.5.
- *make run-g* debe felicitarlo.

Invoke el comando *make zip* para ejecutar todos los tests y generar un archivo *sat.zip* que contiene *sat.c*, con su solución, y *resultados.txt*, con la salida de *make run*, *make run-g*, *make run-thr* y *make run-san*.

Entrega

Ud. solo debe entregar por medio de U-cursos el archivo *sat.zip* generado por *make zip*. Recuerde descargar el archivo que subió, descargar nuevamente los archivos adjuntos y volver a probar la tarea tal cual como la subió a U-cursos. Solo así estará seguro de no haber entregado archivos incorrectos. No se consideran los días de receso, sábados, domingos o festivos.